# AI for Robotics
# Project Milestone 3 Report

Team 2

Winter 2025

## Contents


**1 Introduction** **2**

**2 World Model** **2**
2.1 Rooms and Coordinates . . . . . 2
2.2 Objects and Initial Locations . . . . . 2

**3 Predicates** **3**

**4 Actions** **3**
4.1 Move . . . . . 3
4.2 Pick . . . . . 3
4.3 Place . . . . . 4

**5 Planner Setup** **4**
5.1 Planner and Search Method . . . . . 4
5.2 CLI Workflow . . . . . 4
5.3 Milestone 3 Integration . . . . . 5

**6 ROS 2 Architecture** **5**
6.1 Architecture Overview . . . . . 5
6.2 System Block Diagram . . . . . 6
6.3 Launch Files Description . . . . . 6
6.3.1 Gazebo Simulation Launch (`gazebo.launch.py`) . . . . . 6
6.3.2 Navigation Stack Launch (`navigation2.launch.py`) . . . . . 7
6.4 ROS Topics and Actions . . . . . 7

**7 Evaluation** **7**
7.1 Execution Results . . . . . 7
7.1.1 Navigation Performance . . . . . 7
7.1.2 Manipulation Sequence . . . . . 8

**8 Conclusion** **8**

# 1 Introduction

This report documents the final integration of the *AI for Robotics (MCTR912)* project. While Milestone 2 focused on constructing a high-level symbolic planning system, Milestone 3 focuses on the physical execution of these plans using the TurtleBot3 and OpenManipulator in the Gazebo simulation environment.

The objectives of this final system are:

- **Symbolic Planning:** Maintain the logic-based planner developed in Milestone 2 to generate PDDL plans dynamically.
- **Plan Translation:** Bridge the gap between symbolic PDDL actions (e.g., `move kitchen`) and continuous ROS 2 commands.
- **Physical Execution:** autonomously navigate the robot between rooms using `Nav2` and manipulate objects using the gripper.
- **Closed-Loop Operation:** Update the Knowledge Base (KB) only after successful physical execution of actions in the simulation.

# 2 World Model

## 2.1 Rooms and Coordinates

The environment contains the following rooms, each assigned approximate coordinates (center points) within the Gazebo House World. These coordinates are used by the execution node to send navigation goals to the robot.

- **storage**: $(-4.5, -1.5)$
- **livingroom**: $(-1.0, 1.5)$
- **diningroom**: $(6.0, 1.0)$
- **bathroom**: $(3.0, 5.0)$
- **bedroom**: $(-4.5, 3.5)$
- **kitchen**: $(8.5, -1.5)$

## 2.2 Objects and Initial Locations

The symbolic world maintains the initialization from Milestone 2, with at least eight objects placed in specific rooms:

- cup — kitchen
- pan — kitchen
- plate — diningroom
- notebook — diningroom
- bottle — livingroom
- remote — livingroom
- book — bedroom

- phone — bedroom
- keys — storage
- towel — bathroom

These objects are tracked in the Python Knowledge Base (KB) and updated as the robot physically moves them between rooms.

# 3 Predicates

The following predicates describe the KB state and are shared between Python and PDDL:

- **room(r)** — Room `r` exists in the world.
- **object(o)** — Object `o` exists in the world.
- **robot-at(r)** — Robot is in room `r`.
- **at(o, r)** — Object `o` is in room `r`.
- **handempty()** — Robot is holding nothing.
- **holding(o)** — Robot is holding object `o`.
- **connected(r1, r2)** — Rooms `r1` and `r2` are adjacent.

# 4 Actions

The symbolic action schemas define the logic for the planner. In Milestone 3, each of these actions is mapped to a specific ROS 2 node execution.

## 4.1 Move

- **Parameters:** `from`, `to`
- **Preconditions:** `robot-at(from)`, `connected(from, to)`
- **Effects:** `robot-at(to)`, not `robot-at(from)`
- **ROS Execution:** Triggers `navigation_node.py` with the coordinates of the `to` room.

## 4.2 Pick

- **Parameters:** `o`, `r`
- **Preconditions:** `robot-at(r)`, `at(o, r)`, `handempty`
- **Effects:** `holding(o)`, not `at(o, r)`, not `handempty`
- **ROS Execution:** Triggers `gripper_node.py` with state `"close"`, executing a manipulation sequence (Arm Down → Close → Arm Home).

### 4.3 Place

- **Parameters:** `o`, `r`
- **Preconditions:** `robot-at(r)`, `holding(o)`
- **Effects:** `at(o, r)`, not `holding(o)`, `handempty`
- **ROS Execution:** Triggers `gripper_node.py` with state `"open"`, executing a manipulation sequence (Arm Down → Open → Arm Home).

## 5 Planner Setup

### 5.1 Planner and Search Method

The planner used is **Fast Downward**, executed with A* search and the LM-Cut heuristic:

```
fast-downward --plan-file output/plan.soln \
  domain/domain.pddl problems/problem.pddl \
  --search "astar(lmcut())"
```

### 5.2 CLI Workflow

The system runs an interactive loop via `cli.py`. Below are screenshots from the execution of the planner (Milestone 2 results), demonstrating the system's ability to handle various goal types.

Figure 1: Initial CLI state. The KB shows the robot in the livingroom and the initial locations of all objects.

Figure 2: Successful goal execution: `at cup storage`. The planner generates a sequence to move, pick, and place the cup.

```
Enter goal (or 'quit'): jump to bathroom
Could not understand goal. Examples: 'robot-at kitchen', 'at cup bedroom', '(holding phone)'.

--- Current KB ---
Robot at: storage
Objects: {'pan': 'kitchen', 'plate': 'diningroom', 'notebook': 'diningroom', 'bottle': 'living
room', 'remote': 'livingroom', 'book': 'bedroom', 'phone': 'bedroom', 'keys': 'storage', 'towe
l': 'bathroom', 'cup': 'storage'}
Holding: None

Enter goal (or 'quit'):
```

Figure 3: Invalid goal handling. The system catches the syntax error "jump to bathroom" and suggests valid formats.

```
Enter goal (or 'quit'): robot-at storage
Goal already satisfied. No actions needed.

--- Current KB ---
Robot at: storage
Objects: {'pan': 'kitchen', 'plate': 'diningroom', 'notebook': 'diningroom', 'bottle': 'living
room', 'remote': 'livingroom', 'book': 'bedroom', 'phone': 'bedroom', 'keys': 'storage', 'towe
l': 'bathroom', 'cup': 'storage'}
Holding: None

Enter goal (or 'quit'):
```

Figure 4: Goal already satisfied. The system detects the robot is already at the target location and requires no actions.

```
Enter goal (or 'quit'): quit
daniel-boules@danielboules-ASUS:~/Semester9/AI/Milestone2$
```

Figure 5: Program termination. The user enters 'quit' to exit the loop cleanly.

### 5.3 Milestone 3 Integration

In this final milestone, the loop was extended. Instead of just printing the plan, the CLI now iterates through the generated actions and calls `executor.py` to trigger the physical ROS 2 nodes for each step.

## 6 ROS 2 Architecture

To bridge the symbolic planner with the physical robot, we implemented a modular ROS 2 architecture consisting of a central execution manager and specialized action nodes.

### 6.1 Architecture Overview

The system architecture flow is illustrated in Figure 6. The components operate as follows:

- **Plan Executor (`executor.py`):** Acts as the bridge. It parses PDDL actions (e.g., `move kitchen`) and launches the appropriate ROS 2 nodes.
- **Navigation Node:** Interfaces with the `Nav2` stack. It accepts target coordinates $(x, y)$ and includes a manual tolerance check (0.30m).
- **Gripper Node:** Controls the OpenManipulator. It implements a sequential state machine (Arm Down $\rightarrow$ Actuate $\rightarrow$ Arm Home).

## 6.2 System Block Diagram

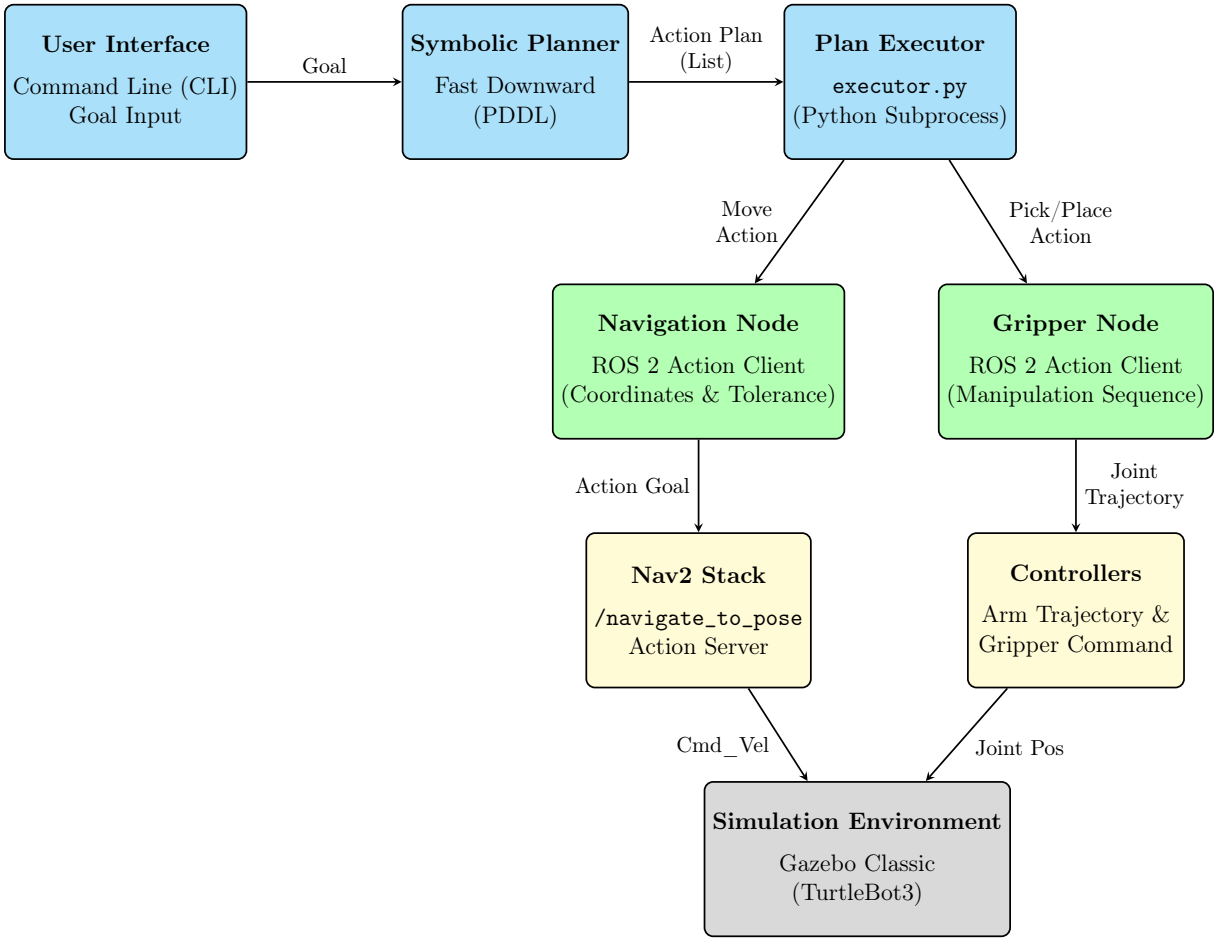



Figure 6: Block diagram illustrating the system architecture and data flow from user input to physical simulation.

## 6.3 Launch Files Description

Two primary launch files orchestrate the simulation and control stack. These files accept various arguments to configure the environment and robot state dynamically.

### 6.3.1 Gazebo Simulation Launch (`gazebo.launch.py`)

This file is responsible for initializing the physical simulation environment.

- **World Loading:** It launches `gazebo_ros` with the `turtlebot3_house.world` file, creating the apartment environment.
- **Robot Spawning:** It utilizes the `spawn_entity.py` node to inject the `turtlebot3_manipulation_system` model into the world.
- **Pose Configuration:** It defines launch arguments `x_pose`, `y_pose`, and `yaw`. For this project, these were adjusted (e.g., $x = 0.0, y = 1.0$) to ensure the robot spawns in a valid starting location within the map.
- **Base Launch:** It includes `base.launch.py`, which likely manages the `robot_state_publisher` to broadcast transforms (TF) for the robot's links.

#### 6.3.2 Navigation Stack Launch (`navigation2.launch.py`)

This file brings up the autonomous navigation capabilities.

- **Nav2 Bringup:** It includes the standard `nav2_bringup` launch file, which starts the AMCL localizer, the Global Planner, and the Local Controller.
- **Map Configuration:** It accepts the `map_yaml_file` argument. We override the default to point to our custom map generated in Milestone 1.
- **Visualization:** It automatically starts `RViz2` with a pre-configured view (`navigation2.rviz`) to visualize the robot's position, the map, and the planned path.
- **Parameters:** It loads the `turtlebot3.yaml` configuration file, which defines critical navigation parameters such as inflation radius and costmap resolution.

### 6.4 ROS Topics and Actions

| **Component** | **Interface Type** | **Topic / Action Name** |
|---|---|---|
| Navigation | Action Client | `/navigate_to_pose` |
| Arm Control | Action Client | `/arm_controller/follow_joint_trajectory` |
| Gripper Control | Action Client | `/gripper_controller/gripper_cmd` |

Table 1: Key ROS 2 interfaces utilized.

## 7 Evaluation

### 7.1 Execution Results

The integrated system was tested in the Gazebo House World. The system successfully performed end-to-end tasks, such as retrieving an object from the kitchen and placing it in the storage room.

#### 7.1.1 Navigation Performance

The robot successfully navigated between rooms using the generated coordinates. Figure 7 illustrates the path planning in RViz overlaid with the physical execution. The usage of the relaxed 0.30m tolerance allowed the robot to reach destinations smoothly without oscillation.

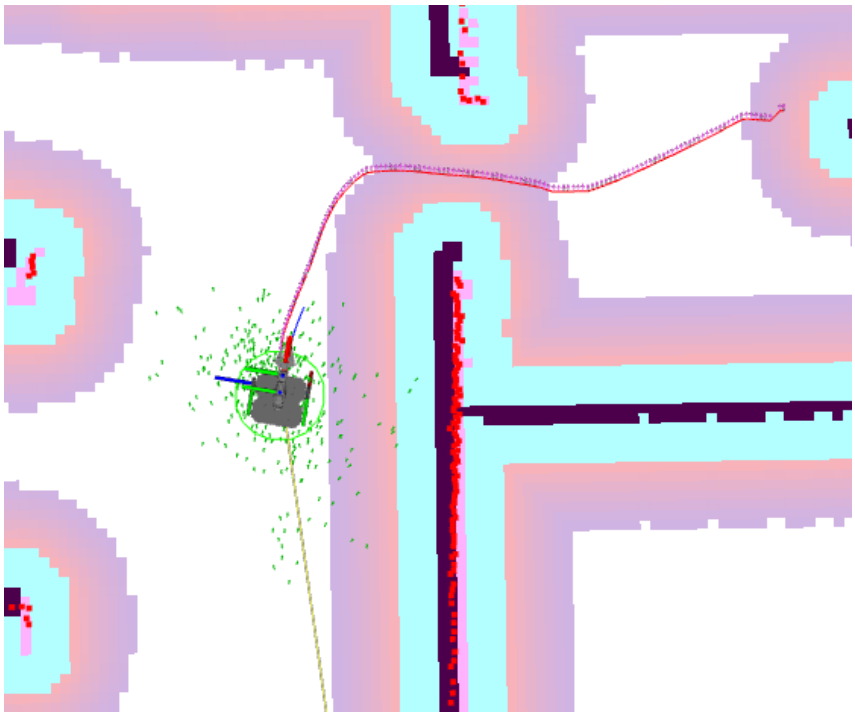
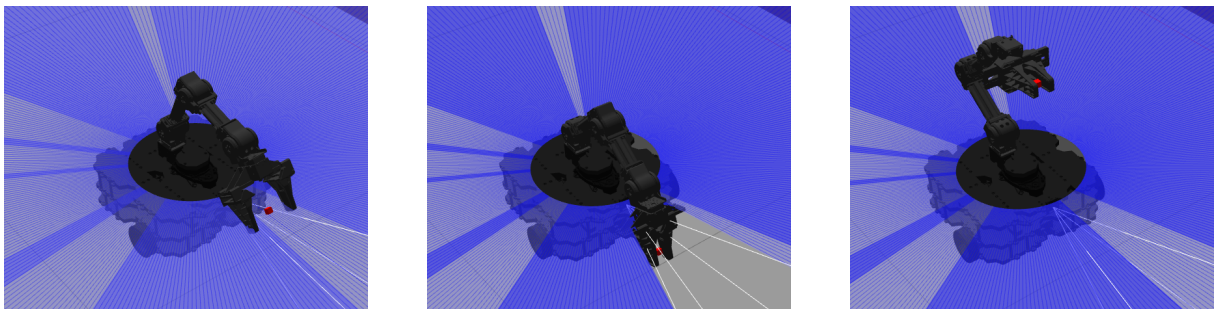


Figure 7: Navigation execution: The Global Planner (**Red Line**) in RViz generated a path from the Dining Room to the Kitchen, and the robot successfully followed the trajectory.

#### 7.1.2 Manipulation Sequence

The manipulation logic was validated by observing the "nodding" sequence. As shown in Figure 8, the robot executes three distinct phases for a `pick` or `place` action:

1. **Approach:** The arm lowers to the pre-set coordinates.
2. **Actuate:** The gripper opens or closes based on the command.
3. **Retract:** The arm returns to the safe driving pose.



(a) Arm Down (b) Gripper Actuation (c) Home Pose

Figure 8: Sequential execution of a manipulation task in Gazebo.

## 8 Conclusion

This project successfully demonstrated the integration of high-level AI planning with low-level robotic control. By connecting a PDDL planner to the ROS 2 ecosystem, we created a robot capable of understanding abstract goals and autonomously executing the necessary physical actions to achieve them.